Great question.

When you see:

```
type Storage struct {
   Db *sql.DB
}
```

or simply

```
Db *sql.DB
```

it means:

- `Db` → the field name.
- `*sql.DB` → the field's type.

Let's break it down.

`sql.DB`

`sql.DB` is a type provided by Go's standard library package:

```
import "database/sql"
```

It represents **a database handle**, not a single database connection.

Think of it like this:

```
sql.DB
  │
  ├── Connection 1
  ├── Connection 2
  ├── Connection 3
  └── ...
```

It manages a pool of connections for you.

Why `*sql.DB`?

The * means **pointer**.

So:

Db *sql.DB

means:

> "Db stores a pointer to a `sql.DB` object."

You almost always use `*sql.DB`, not `sql.DB`, because:

- `sql.DB` is a large struct.
- It is designed to be shared.
- Copying it is incorrect.
- Most functions like `sql.Open()` return `*sql.DB`.

Example:

db, err := sql.Open("sqlite3", "students.db")

Here,

db

is of type:

*sql.DB

Example

```
type Storage struct {
    Db *sql.DB
}
```

Create it:

```
db, err := sql.Open("sqlite3", "students.db")
if err != nil {
    log.Fatal(err)
```

```
}

storage := Storage{
    Db: db,
}
```

Now you can use it:

```
rows, err := storage.Db.Query("SELECT * FROM students")
```

Real-world analogy

Imagine a company.

- `sql.DB` = the **office reception** that manages all employees (connections).
- Individual database connections are the employees.
- When you call `Query()`, the reception gives you an available employee to do the work and then takes them back when finished.

That's why `sql.DB` is called a **database handle** or **connection pool**, not a single connection.

Common misconception

Many beginners think:

```
db *sql.DB
```

means "one database connection."

It **does not**.

It actually represents a **pool of reusable database connections**, which Go manages automatically. This is why you generally create one `*sql.DB` for your application and reuse it everywhere.

Excellent observation. This is exactly the kind of question you should ask when learning databases.

The answer is **yes**—SQLite is automatically generating the `id` because of this line in your table definition:

id INTEGER PRIMARY KEY AUTOINCREMENT

Let's break it down.

When the table is created

```
CREATE TABLE students(
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   email TEXT,
   age INTEGER
)
```

Here you're telling SQLite:

- `INTEGER` → the column stores integers.
- `PRIMARY KEY` → this is the unique identifier for each row.
- `AUTOINCREMENT` → SQLite should automatically assign the next available ID whenever a new row is inserted.

Now look at your INSERT statement

```
stmt, err := s.Db.Prepare(
   "INSERT INTO students(name,email,age) VALUES (?,?,?)",
)
```

Notice that **`id` is not included**.

When you execute:

result, err := stmt.Exec(name, email, age)

SQLite internally does something like this:

INSERT INTO students(id, name, email, age)
VALUES (1, 'Suyash', 'suyash@gmail.com', 23)

For the next insert:

INSERT INTO students(id, name, email, age)
VALUES (2, 'Rahul', 'rahul@gmail.com', 21)

Then:

INSERT INTO students(id, name, email, age)
VALUES (3, 'Priya', 'priya@gmail.com', 20)

You never have to provide the ID yourself.

How do you get the generated ID?

That's exactly what this code is doing:

lastId, err := result.LastInsertId()

Suppose the new student gets ID 5.

Then:

fmt.Println(lastId)

prints:

5

So your function:

func (s *Sqlite) CreateStudent(...) (int64, error)

returns the newly created student's ID.

What if you remove `AUTOINCREMENT`?

If you write:

id INTEGER PRIMARY KEY

SQLite still automatically generates IDs in most cases because an `INTEGER PRIMARY KEY` is an alias for SQLite's internal `ROWID`.

The difference is subtle:

- `INTEGER PRIMARY KEY` → automatically assigns IDs, and may reuse some deleted IDs in certain situations.
- `INTEGER PRIMARY KEY AUTOINCREMENT` → guarantees that automatically generated IDs are always larger than any previously generated ID and won't reuse old ones, at the cost of a little extra bookkeeping.

For many applications, `INTEGER PRIMARY KEY` is sufficient. `AUTOINCREMENT` is only needed if you specifically require never reusing an automatically generated ID.

Your code is correct

Your insert flow is exactly how it should be:

stmt.Exec(name, email, age)

⬇️ SQLite generates the ID.

⬇️

result.LastInsertId()

⬇️

You return the generated ID to the caller.

This is the standard pattern used with SQLite and many other SQL databases.